

IDS568 Milestone 5: LLM Inference Optimization

1. Introduction

This project focuses on optimizing large language model inference using dynamic batching and caching techniques. The objective is to improve latency, throughput, and system efficiency under concurrent request loads.

The system was implemented using FastAPI, Hugging Face Transformers, and asynchronous request handling. Performance improvements were evaluated using controlled benchmark experiments.

2. System Architecture

The system consists of:

- FastAPI inference server
- Dynamic batching module
- In-memory caching layer
- Benchmarking suite
- Performance visualization pipeline

Requests follow this pipeline:

Client → FastAPI → Cache → Batchers → Model → Response

3. Experimental Setup

Model: facebook/opt-125m

Framework: FastAPI + PyTorch

Hardware: MacBook Air CPU

Concurrency: 5, 20, 50 requests

Benchmarks performed:

- Cold vs Warm cache
 - Batching latency
 - Load level comparison
 - Cache hit rate tracking
-

4. Results

Cold vs Warm Cache

Cold cache latency averaged 646 ms, while warm cache latency dropped to under 1 ms. This demonstrates significant performance improvements through caching repeated prompts.

Batching Performance

Batching improved throughput under concurrent load. Requests processed together reduced compute overhead and improved efficiency.

Load Testing

Low load (5 concurrent)
Average latency: 755 ms

Medium load (20 concurrent)
Average latency: 2.8 ms

High load (50 concurrent)
Average latency: 6.7 ms

Cache hit rate increased with repeated prompts.

5. Trade-off Analysis

Batching improves throughput but increases wait time for individual requests.

Caching improves latency for repeated prompts but provides limited benefit for unique prompts.

Aggressive batching may increase tail latency.

6. Scaling Strategy

Future improvements include:

- Redis distributed caching
 - GPU inference deployment
 - Continuous batching
 - Autoscaling based on queue length
-

7. Conclusion

Dynamic batching and caching significantly improved inference performance. The optimized system demonstrated strong performance under concurrent load while maintaining low latency and high cache hit rates.